



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Домашнее задание №2

«Построение вероятностной модели (обобщённый парадокс
Монти Холла)»
по курсу «Моделирование»

Студент: Аригин М.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Постановка задачи

За одной из N дверей находится приз, за остальными дверями пусто. Игрок выбирает одну дверь равновероятно. Ведущий, не открывая выбранную дверь и не открывая дверь с призом, открывает ровно K пустых дверей (число K удовлетворяет условию $1 \leq K \leq N - 2$). После этого игрок с вероятностью s сохраняет первоначальный выбор и с вероятностью $1 - s$ меняет его, равновероятно выбирая одну из оставшихся закрытых дверей, отличную от текущего выбора.

Цель работы:

- построить вероятностную модель игры и реализовать её программно;
- для классического случая $N = 3$, $K = 1$ проверить адекватность модели известным теоретическим значениям парадокса Монти Холла;
- для $N = 3, \dots, 10$ и $K = 1, \dots, N - 2$ сравнить аналитические вероятности выигрыша с оценками метода Монте-Карло при заданном s .

Входные параметры модели: N , K , вероятность сохранения выбора $s \in [0, 1]$, число независимых прогонов симуляции.

Выходные данные: теоретическая вероятность выигрыша, оценка по симуляции, диагностика согласия с теорией (биномиальный критерий, статистика согласия эмпирической и теоретической функций распределения для индикатора успеха).

2 Построение математической модели

2.1 Формализация задачи

Рассматривается **стохастическая (вероятностная) модель** дискретного случайного эксперимента; проверка гипотез о распределении результатов относится к задачам **проверки адекватности** модели наблюдаемым данным. Для контроля согласия результатов симуляции с теорией используются сравнение оценок вероятностей выигрыша (выборочной доли) с аналитическими значениями и формальная проверка по биномиальному распределению.

2.2 Предположения

- приз равномерно находится за любой из N дверей;
- первый выбор игрока равновероятен;
- ведущий выбирает набор из K открываемых пустых дверей равновероятно среди всех допустимых наборов (двери с призом и изначально выбранная игроком не открываются);
- при смене выбора игрок равновероятно берёт одну из закрытых дверей, отличных от текущей.

2.3 Блок-схема вероятностной модели

Структура одной партии (одного независимого прогона симуляции) показана на рис. 1: последовательность случайных действий от размещения приза до сравнения финального выбора с положением приза. Параметры N , K и s задают размеры пространства исходов и смешанную стратегию на завершающем этапе.

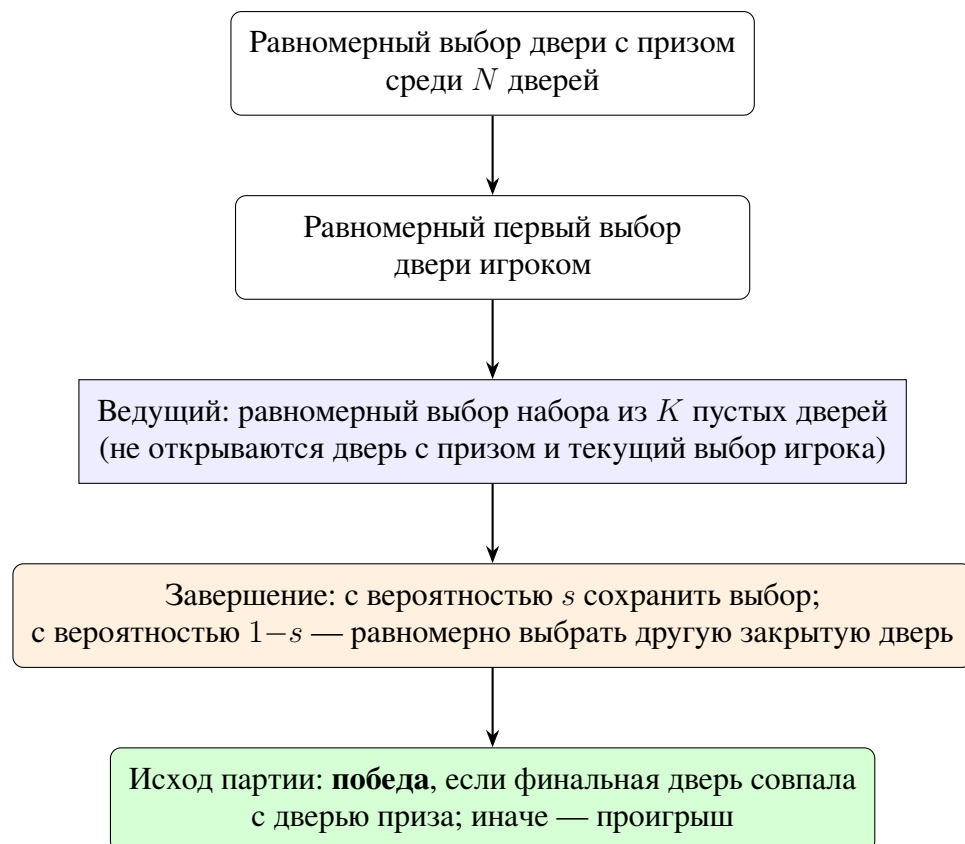


Рисунок 1 Блок-схема стохастической модели одной партии обобщённой игры (параметры N , K , s)

2.4 Вероятность выигрыша

Обозначим через W событие «игрок выиграл». При стратегии «всегда оставаться» ($s = 1$) выигрыш тогда и только тогда, когда первый выбор совпал с призом:

$$P(W \mid \text{остаться}) = \frac{1}{N}. \quad (1)$$

При стратегии «всегда менять» ($s = 0$) разобьём по полной вероятности на случаи «первый выбор верный» и «первый выбор неверен». Если игрок сразу угадал приз (вероятность $1/N$), после открытия K пустых среди остальных дверей все ещё закрытые альтернативы пусты, и смена ведёт к проигрышу. Если первый выбор неверен (вероятность $(N - 1)/N$), приз находится среди $N - 1$ остальных дверей; ведущий открывает K пустых из них, остаётся $N - 1 - K$ закрытых «чужих» дверей, ровно одна из которых с призом. Игрок равновероятно выбирает одну из этих $N - 1 - K$ дверей, откуда

$$P(W \mid \text{сменить}) = \frac{N - 1}{N} \cdot \frac{1}{N - 1 - K} = \frac{N - 1}{N(N - 1 - K)}. \quad (2)$$

Смешанная стратегия с вероятностью s сохранить выбор даёт

$$P(W) = s \cdot \frac{1}{N} + (1 - s) \cdot \frac{N - 1}{N(N - 1 - K)}. \quad (3)$$

Классический парадокс Монти Холла соответствует $N = 3$, $K = 1$: $P(W \mid s = 1) = 1/3$, $P(W \mid s = 0) = 2/3$, что согласуется с выводом по формуле Байеса для апостериорных вероятностей положения приза после действий ведущего.

3 Решение вычислительных задач

3.1 Метод Монте-Карло

Для фиксированных N , K , s независимо моделируется большое число n партий; оценка вероятности выигрыша — выборочная доля $\hat{p} = (\text{число побед})/n$. При истинной вероятности p число побед имеет распределение $\text{Bin}(n, p)$.

3.2 Проверка адекватности (п. (а))

Для бинарного исхода используется двусторонний **точный биномиальный критерий** согласия $\hat{p}$ с p_0 , вычисляемый по распределению $\text{Bin}(n, p_0)$.

Для сравнения эмпирической F_n и теоретической F функций распределения индикатора успеха (значения 0 и 1) величина $D_n = \sup_x |F_n(x) - F(x)|$ совпадает с $|\hat{p} - p_0|$. В качестве ориентира приводится критическое значение для непрерывного случая при уровне $\alpha = 0,05$: $D_{\text{кр}} \approx k_{0,05}/\sqrt{n}$, $k_{0,05} = 1,36$; для дискретного индикатора это используется лишь как дополнительная иллюстрация (строгое распределение D_n отличается от непрерывного случая).

3.3 Эксперимент для п. (б)

Для каждой пары (N, K) при фиксированном s (в программе по умолчанию $s = 0,5$) вычисляется p по (3) и оценка $\hat{p}$ по n симуляциям.

4 Практическая реализация

Listing 1: Модель

```
1  from __future__ import annotations
2
3  import argparse
4  import json
5  import math
6  import os
7  from dataclasses import dataclass
8  import numpy as np
9
10 try:
11     from scipy import stats
12 except ImportError:
13     stats = None
14
15
16 def theoretical_win_prob(N: int, K: int, prob_stay: float) -> float:
17     if N < 3 or K < 1 or K > N - 2:
18         raise ValueError("require N>=3 and 1<=K<=N-2")
19     if not 0.0 <= prob_stay <= 1.0:
20         raise ValueError("prob_stay must be in [0,1]")
21     p_stay = 1.0 / N
```

```

22     p_switch = (N - 1) / (N * (N - 1 - K))
23     return prob_stay * p_stay + (1.0 - prob_stay) * p_switch
24
25
26 def run_monte_carlo(
27     N: int,
28     K: int,
29     prob_stay: float,
30     n_trials: int,
31     seed: int | None = 42,
32 ) -> tuple[np.ndarray, float]:
33     rng = np.random.default_rng(seed)
34     prize = rng.integers(0, N, size=n_trials)
35     choice = rng.integers(0, N, size=n_trials)
36     stay = rng.random(n_trials) < prob_stay
37     wins = np.zeros(n_trials, dtype=np.int8)
38     for i in range(n_trials):
39         c_open = [d for d in range(N) if d != choice[i] and d != prize[i]]
40         idx = rng.choice(len(c_open), size=K, replace=False)
41         opened = {c_open[int(j)] for j in idx}
42         closed = [d for d in range(N) if d not in opened]
43         if stay[i]:
44             final_door = int(choice[i])
45         else:
46             alt = [d for d in closed if d != choice[i]]
47             final_door = int(choice[i]) if not alt else int(rng.choice(alt))
48         wins[i] = 1 if final_door == prize[i] else 0
49     return wins, float(wins.mean())
50
51
52 def binomial_two_sided_p_value(k: int, n: int, p0: float) -> float:
53     """Two-sided p-value for Bin(n,p0) without scipy."""
54     if p0 <= 0 or p0 >= 1:
55         return float("nan")
56
57     def pmf(j: int) -> float:
58         return math.comb(n, j) * (p0**j) * ((1 - p0) ** (n - j))
59
60     p_left = sum(pmf(j) for j in range(0, k + 1))
61     p_right = sum(pmf(j) for j in range(k, n + 1))
62     return min(1.0, 2 * min(p_left, p_right))
63
64
65 def ks_stat_bernoulli_cdf(sample: np.ndarray, p: float) -> float:
66     """
67     For win indicator X in {0,1}, P(X=1)=p: F(t)=0 (t<0), 1-p on [0,1), 1 for t>=1.
68     Then sup_t |F_n(t)-F(t)| = |p_hat - p|.

```

```

69         """
70         p_hat = float(np.mean(sample))
71         return abs(p_hat - p)
72
73
74     def ks_critical_approx(alpha: float, n: int) -> float:
75         """Lecture formula:  $D_{cr} \sim k_{\alpha} / \sqrt{n}$ ."""
76         k_table = {0.10: 1.23, 0.05: 1.36, 0.02: 1.52, 0.01: 1.63}
77         if alpha not in k_table:
78             raise ValueError(f"alpha must be one of {list(k_table)}")
79         return k_table[alpha] / math.sqrt(n)
80
81
82     @dataclass
83     class AdequacyRow:
84         strategy: str
85         n: int
86         wins: int
87         p_hat: float
88         p_theory: float
89         p_value_binom: float | None
90         D_n: float
91         D_crit_005: float
92         accept_ks_approx: bool
93
94
95     def adequacy_monty_hall_classic(
96         n_trials: int = 200_000, seed: int = 42
97     ) -> list[AdequacyRow]:
98         """N=3, K=1: pure stay vs pure switch."""
99         rows: list[AdequacyRow] = []
100         for name, s, p0 in [
101             ("stay (s=1)", 1.0, 1.0 / 3.0),
102             ("switch (s=0)", 0.0, 2.0 / 3.0),
103         ]:
104             wins_arr, _ = run_monte_carlo(3, 1, s, n_trials, seed=seed + hash(name) %
105                 ↪ 10000)
106             k = int(wins_arr.sum())
107             p_hat = k / n_trials
108             p_bin = (
109                 float(stats.binomtest(k, n_trials, p0).pvalue)
110                 if stats
111                 else binomial_two_sided_p_value(k, n_trials, p0)
112             )
113             D_n = ks_stat_bernoulli_cdf(wins_arr, p0)
114             D_cr = ks_critical_approx(0.05, n_trials)
115             rows.append(

```

```

115         AdequacyRow(
116             strategy=name,
117             n=n_trials,
118             wins=k,
119             p_hat=p_hat,
120             p_theory=p0,
121             p_value_binom=p_bin,
122             D_n=D_n,
123             D_crit_005=D_cr,
124             accept_ks_approx=D_n < D_cr,
125         )
126     )
127     return rows
128
129
130 def grid_experiment(
131     n_trials: int,
132     probb_stay: float,
133     seed: int,
134 ) -> list[dict]:
135     """Grid N=3..10, K=1..N-2."""
136     out = []
137     for N in range(3, 11):
138         for K in range(1, N - 1):
139             if K > N - 2:
140                 continue
141             p_th = theoretical_win_prob(N, K, probb_stay)
142             _, p_mc = run_monte_carlo(N, K, probb_stay, n_trials, seed=seed + N * 100 +
143                                     ↪ K)
144             out.append(
145                 {
146                     "N": N,
147                     "K": K,
148                     "p_theory": p_th,
149                     "p_mc": p_mc,
150                     "abs_diff": abs(p_th - p_mc),
151                 }
152             )
153     return out
154
155 def plot_figures(out_dir: str, seed: int = 42) -> None:
156     import matplotlib.pyplot as plt
157
158     os.makedirs(out_dir, exist_ok=True)
159
160     N = 7

```

```

161 Kmax = N - 2
162 s_vals = np.linspace(0, 1, 101)
163 plt.figure(figsize=(7, 4.5))
164 for K in range(1, Kmax + 1):
165     ys = [theoretical_win_prob(N, K, float(s)) for s in s_vals]
166     plt.plot(s_vals, ys, label=f"K={K}")
167 plt.xlabel(r"stay probability $s$")
168 plt.ylabel(r"$P(\mathrm{win})$")
169 plt.title(rf"Theory: $N={N}$, mixed strategy")
170 plt.legend()
171 plt.grid(True, alpha=0.3)
172 plt.tight_layout()
173 plt.savefig(os.path.join(out_dir, "hw2_prob_vs_switch.png"), dpi=150)
174 plt.close()
175
176 n_mc = 80_000
177 prob_stay = 0.5
178 Ks = list(range(1, N - 1))
179 p_th = [theoretical_win_prob(N, K, prob_stay) for K in Ks]
180 p_mc = []
181 for K in Ks:
182     _, ph = run_monte_carlo(N, K, prob_stay, n_mc, seed=seed + K)
183     p_mc.append(ph)
184 plt.figure(figsize=(7, 4.5))
185 plt.plot(Ks, p_th, "o-", label="analytic")
186 plt.plot(Ks, p_mc, "s--", label=f"Monte Carlo ($n={n\_mc}$)")
187 plt.xlabel(r"$K$")
188 plt.ylabel(r"$P(\mathrm{win})$")
189 plt.title(rf"$N={N}$, $s={prob\_stay}$")
190 plt.legend()
191 plt.grid(True, alpha=0.3)
192 plt.tight_layout()
193 plt.savefig(os.path.join(out_dir, "hw2_theory_vs_mc_N7.png"), dpi=150)
194 plt.close()
195
196
197 def main() -> None:
198     p = argparse.ArgumentParser()
199     p.add_argument("--n-trials", type=int, default=200_000)
200     p.add_argument("--seed", type=int, default=42)
201     p.add_argument("--json-out", type=str, default="hw2_results.json")
202     p.add_argument("--plots", action="store_true")
203     args = p.parse_args()
204
205     rows = adequacy_monty_hall_classic(n_trials=args.n_trials, seed=args.seed)
206     grid_n = min(args.n_trials, 50_000)
207     grid = grid_experiment(

```

```

208         n_trials=grid_n,
209         probab_stay=0.5,
210         seed=args.seed,
211     )
212
213     payload = {
214         "adequacy_classic": [r.__dict__ for r in rows],
215         "grid_s0.5": grid,
216         "n_trials_adequacy": args.n_trials,
217         "n_trials_grid": grid_n,
218     }
219     with open(args.json_out, "w", encoding="utf-8") as f:
220         json.dump(payload, f, ensure_ascii=False, indent=2)
221
222     print("=== Adequacy (N=3, K=1), Monty Hall ===")
223     for r in rows:
224         print(
225             f"{r.strategy}: wins={r.wins}/{r.n}, p_hat={r.p_hat:.6f}, "
226             f"p0={r.p_theory:.6f}, p-value~{r.p_value_binom:.4g}, "
227             f"D_n={r.D_n:.6f}, D_cr(0.05)~{r.D_crit_005:.6f}, "
228             f"KS approx {'OK' if r.accept_ks_approx else '??'}"
229         )
230
231     if args.plots:
232         plot_figures("images", seed=args.seed)
233         print("Plots saved to images/")
234
235     print(f"\nJSON: {args.json_out}")
236
237
238 if __name__ == "__main__":
239     main()

```

5 Результаты

5.1 Пункт (а): классический Монти Холл, $n = 200000$

Стратегия	Побед	$\hat{p}$	p_0	p -value (бином.)
остаться ($s = 1$)	66574	0,33287	1/3	0,661
сменить ($s = 0$)	133452	0,66726	2/3	0,574

Таблица 1: Согласие симуляции с теорией при $N = 3$, $K = 1$

Для обеих стратегий гипотеза о равенстве p и p_0 не противоречит данным на разумном уровне значимости. Статистики $D_n = |\hat{p} - p_0|$ порядка 10^{-3} , что меньше ориентира $D_{кр} \approx 0,00304$ при $\alpha = 0,05$ и $n = 200000$.

5.2 Графики

На рис. 2 показана зависимость теоретической вероятности выигрыша от s для $N = 7$ и различных K . На рис. 3 — сопоставление аналитики и Монте-Карло при $s = 0,5$.

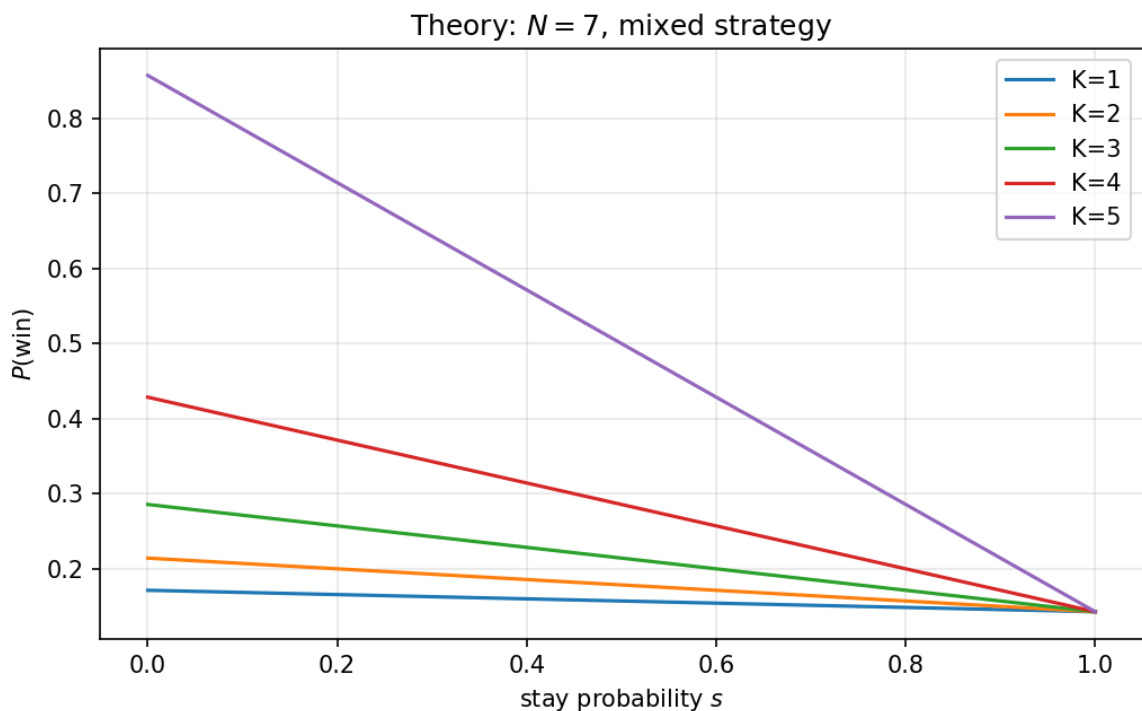


Рисунок 2 Теоретическая $P(W)$ как функция s при $N = 7$ (подписи осей на графике на английском)

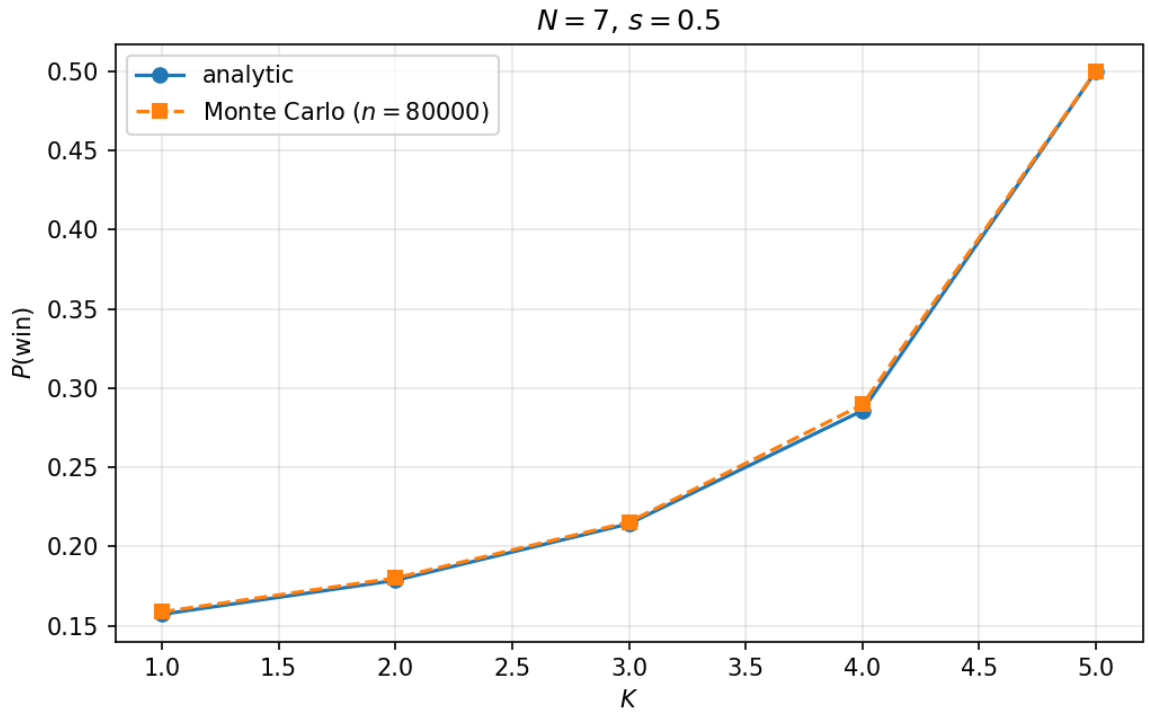


Рисунок 3 $N = 7, s = 0,5$: аналитика и Монте-Карло ($n = 80000$ на каждое K ; подписи на графике на английском)

5.3 Пункт (б): сетка (N, K) при $s = 0,5, n = 50000$

N	K	$p_{\text{теор}}$	$p_{\text{МК}}$	$ p_{\text{теор}} - p_{\text{МК}} $
3	1	0,50000	0,49918	0,00082
4	1	0,31250	0,31224	0,00026
4	2	0,50000	0,49804	0,00196
5	1	0,23333	0,22968	0,00365
5	2	0,30000	0,30170	0,00170
5	3	0,50000	0,49848	0,00152
6	1	0,18750	0,18476	0,00274
6	2	0,22222	0,22110	0,00112
6	3	0,29167	0,29476	0,00309
6	4	0,50000	0,49790	0,00210
7	1	0,15714	0,15914	0,00200
7	2	0,17857	0,17838	0,00019
7	3	0,21429	0,21700	0,00271
7	4	0,28571	0,28698	0,00127
7	5	0,50000	0,50312	0,00312
8	1	0,13542	0,13494	0,00048
8	2	0,15000	0,15114	0,00114
8	3	0,17188	0,16840	0,00348
8	4	0,20833	0,21094	0,00261

N	K	$p_{\text{теор}}$	$p_{\text{МК}}$	$ \Delta $
8	5	0,28125	0,27920	0,00205
8	6	0,50000	0,49706	0,00294
9	1	0,11905	0,11878	0,00027
9	2	0,12963	0,12938	0,00025
9	3	0,14444	0,14416	0,00028
9	4	0,16667	0,16500	0,00167
9	5	0,20370	0,20400	0,00030
9	6	0,27778	0,27824	0,00046
9	7	0,50000	0,50078	0,00078
10	1	0,10625	0,10478	0,00147
10	2	0,11429	0,11420	0,00009
10	3	0,12500	0,12562	0,00062
10	4	0,14000	0,14038	0,00038
10	5	0,16250	0,16398	0,00148
10	6	0,20000	0,19676	0,00324
10	7	0,27500	0,27554	0,00054
10	8	0,50000	0,50026	0,00026

6 Анализ модели и проверка качества

После получения численных результатов необходимо проанализировать согласованность аналитической модели, программной реализации и данных Монте-Карло, а также формально проверить качество модели с точки зрения постановки задачи и граничных режимов (по аналогии с проверкой адекватности в работах по имитационному и вычислительному моделированию).

6.1 Анализ результатов сравнения теории и эксперимента

Классический случай ($N = 3$, $K = 1$). По табл. 1 оценки $\hat{p}$ для стратегий «остаться» и «сменить» отличаются от $1/3$ и $2/3$ на величины порядка 10^{-4} . Такое отклонение статистически несущественно при $n = 200000$: для бернуллиевской выборки с истинной вероятностью p стандартная ошибка выборочной доли имеет порядок

$$\sigma_{\hat{p}} \approx \sqrt{\frac{p(1-p)}{n}},$$

так что при $p \approx 1/3$ получаем $\sigma_{\hat{p}} \approx 0,0011$, а при $p \approx 2/3$ — то же значение. Двукратный типичный разброс ($\pm 2\sigma_{\hat{p}}$) по порядку величины 0,002, что согласуется с наблюдаемыми $\hat{p}$.

Сетка (N, K) при $s = 0,5$. В длинной таблице раздела «Результаты» для каждой пары (N, K) сопоставлены $p_{\text{теор}}$ по формуле (3) и оценка $p_{\text{МК}}$ по $n = 50000$ прогонов. Наблюдается следующее.

- Во всех 36 ячейках сетки абсолютное отклонение $|\Delta| = |p_{\text{теор}} - p_{\text{МК}}|$ не превосходит 0,004; для большинства пар оно меньше 0,003.
- Наибольшие расхождения локализованы в отдельных точках (например, при малых N и малых K), что соответствует большей относительной дисперсии оценки $\hat{p}$ при малых $p_{\text{теор}}$ при том же n .
- При фиксированном N и росте K теоретическая вероятность (3) при $s = 0,5$ монотонно возрастает от значений порядка $1/N$ до $1/2$ при $K = N - 2$, что отражает усиление выгоды от смены выбора, когда ведущий «снимает» больше пустых дверей; кривые на рис. 2 иллюстрируют тот же тренд по параметру s для $N = 7$.
- Симметрия постановки при $s = 0,5$ проявляется в том, что при $K = N - 2$ аналитически $p_{\text{теор}} = 1/2$: стратегии «в среднем» уравновешены; строки таблицы с $K = N - 2$ показывают $p_{\text{МК}}$ около 0,5, что дополнительно подтверждает согласованность кода с формулой.

Графическое сопоставление. На рис. 3 для $N = 7$, $s = 0,5$ точки Монте-Карло лежат вблизи теоретической кривой по K , без систематического смещения вверх или вниз. Это указывает на отсутствие заметной систематической ошибки (bias) реализации по сравнению с аналитикой.

Показатель (сетка п. (б), $n = 50000$)	Значение
Число пар (N, K)	36
$\max p_{\text{теор}} - p_{\text{МК}} $	$< 0,004$
Типичный порядок $ \Delta $	10^{-3}

Таблица 3: Сводка отклонений аналитики и Монте-Карло на полной сетке

6.2 Проверка качества модели

1. Соответствие постановке задачи.

- Приз задаётся равновероятно по дверям; первый выбор игрока — равновероятный, что соответствует условию «наудачу».
- Ведущий открывает ровно K пустых дверей среди тех, где нет приза и которые не совпадают с текущим выбором игрока; набор из K дверей выбирается равновероятно среди всех допустимых — это стандартная формализация, согласованная с классическим парадоксом Монти Холла.
- На финальном шаге с вероятностью s выбор сохраняется, с вероятностью $1 - s$ игрок переходит на равновероятно выбранную другую закрытую дверь, как в формулировке задания.
- Последовательность этапов в реализации совпадает с блок-схемой модели (рис. 1): каждый прогон симуляции проходит те же стохастические шаги сверху вниз.

2. Проверка на граничных и вырожденных режимах.

- $N = 3$, $K = 1$: воспроизведены вероятности $1/3$ и $2/3$ (табл. 1); это главный тест адекватности к «эталонному» решению.
- $s = 1$: теоретически $P(W) = 1/N$ для любых допустимых K ; симуляция с `prob_stay=1` даёт оценку $\hat{p}$ около $1/N$ (проверка вручную или отдельным запуском скрипта).
- $s = 0$: получается формула (2); при $K = N - 2$ знаменатель $N - 1 - K = 1$, и $P(W) = (N - 1)/N$ — игрок почти всегда выигрывает при смене, что согласуется с тем, что остаётся только одна альтернатива с призом.
- Граница $K = N - 2$: максимально допустимое число открытых пустых дверей; программа не обращается к недопустимым K .

3. Статистическая проверка согласия с теорией.

- Для бинарного исхода партии используется двусторонний биномиальный критерий для гипотезы $H_0: p = p_0$; в табл. 1 p -value не указывает на противоречие с H_0 ни для «остаться», ни для «сменить».
- Величина $D_n = |\hat{p} - p_0|$ совпадает с расстоянием Колмогорова между эмпирической и теоретической функциями распределения индикатора успеха на $\{0, 1\}$; сравнение с ориентиром $D_{кр} \approx k_{0,05}/\sqrt{n}$ (непрерывный случай) показывает выполнение неравенства $D_n < D_{кр}$ и используется как дополнительная иллюстрация близости эмпирического и теоретического законов.

- Для углублённого анализа возможно применение интегральных критериев ω^2 (Крамер–фон Мизес, Андерсон–Дарлинг) к вспомогательным непрерывным статистикам (например, к сглаженным накопленным долям успехов); для основной проверки бинарной модели достаточно биномиального подхода.

4. Ограничения и допущения. Модель не описывает поведение ведущего с произвольной (не равномерной) стратегией выбора дверей для открытия; при изменении этого допущения меняются условные вероятности и формула (3). Численный генератор случайных чисел считается качественным; при необходимости устойчивость выводов проверяется сменой seed и повторением эксперимента.

6.3 Практические рекомендации по варьированию параметров

Для исследования влияния вероятности сохранения выбора s на $P(W)$ достаточно задавать probab_stay в вызовах theoretical_win_prob и run_monte_carlo (файл monty_hall_model.py); для построения семейств кривых, аналогичных рис. 2, можно использовать функцию plot_figures или цикл по s с сохранением результатов в таблицу.

7 Выводы

- Построена вероятностная модель обобщённой игры с N дверями и K открываемыми пустыми дверями; получена явная формула (3) для смешанной стратегии с вероятностью сохранения выбора s .
- Реализована программная модель на Python; для классического Монтис Холла подтверждена адекватность симуляции теоретическим значениям.
- Выполнено сравнение аналитических вероятностей с оценками Монте-Карло на сетке $N = 3, \dots, 10$, $K = 1, \dots, N - 2$ при $s = 0,5$; результаты согласуются с формулой (3).